

1.5em Opt

Retrosynthesis - Reaction Center Classification

Πέτρος Τριαντάφυλλος, mtn2419
Αχιλλέας Οικονομόπουλος, mtn24

Ιούλιος 2025



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
UNIVERSITY OF PIRAEUS



ΕΘΝΙΚΟ ΚΕΝΤΡΟ ΕΡΕΥΝΑΣ
ΦΥΣΙΚΩΝ ΕΠΙΣΤΗΜΩΝ «ΔΗΜΟΚΡΙΤΟΣ»

Class: Deep Learning
Professor: Γ. Μπουρίτσας

Contents

1	Introduction	4
2	Data and Representation	4
2.1	Dataset	4
2.1.1	Data Cleaning	5
2.1.2	RDKit	5
2.2	Graph Representation	6
2.3	Feature Engineering & Labeling	6
2.4	Key Challenges	7
3	Architecture	7
3.1	Algorithm	7
3.1.1	The Model	9
3.1.2	Beam Search Algorithm	10
3.1.3	Frameworks, Techniques, Metrics used	11
4	Benchmarking	12
4.1	Baselines	12
4.1.1	A weaker model	12
4.1.2	G2G Center Identification	12
4.2	Metrics	13
4.3	Training-Validation-Test pipeline	13
5	Results	14
5.1	Benchmarks	14
5.2	Baseline	14
5.3	Our Best Model	15
5.3.1	Final parameters	15
6	Experimentation	20
6.1	Ablations	20
6.1.1	Beam Search	20
6.1.2	No global features (& Multi-task)	20
6.1.3	Number of Hydrogens	21
7	Conclusion	21
7.1	Future Improvements	21
8	References	22

1 Introduction

Retrosynthesis is the process of deconstructing a target molecule into simpler precursor structures by reversing synthetic reactions. Pioneered by E.J. Corey, this “reverse” problem-solving approach is fundamental in organic synthesis. By repeatedly breaking down molecules step by step, one can trace a pathway from the final product back to commercially available or at the very least simpler starting materials, thereby mapping out a synthetic route. This task is inherently more complex than forward reaction prediction. While forward prediction is often a one-to-one mapping from a set of reactants to a well-defined product, retrosynthesis is a one-to-many problem, where a single product can potentially be synthesized through a multitude of reaction pathways. The sheer scale of the known chemical space makes this a formidable challenge for humans and a prime candidate for computational assistance.

Single-step retrosynthesis methods fall into three main categories: template-based, template-free, and semi-template.

- Template-based approaches match products to a fixed library of known reaction rules. While highly interpretable, they fail to generalize to novel reactions.
- Template-free models, such as sequence-to-sequence Transformers, learn directly from data to translate products into reactants. This offers greater flexibility but offer little to none interpretability and often produces chemically invalid or non-plausible molecules if not carefully constrained.
- Semi-template frameworks offer a balance by breaking the problem into two intuitive steps, mimicking a chemist’s logic. They first predict the reaction center (where the reaction occurs) and then complete the resulting molecular fragments (synthons) into valid reactants. This hybrid approach offers both robust chemical plausibility and readily available interpretability of predictions.

In our work, we focus specifically on reaction center prediction as a standalone task.

Reaction center prediction means determining which bond(s) in the target product are the ones that were formed in the forward reaction (and hence should be broken in retrosynthesis). Correctly identifying the reaction center is critical – it essentially sets the stage for the entire retrosynthetic step. Once the right bond is cut, the resulting pieces (often termed synthons) can be mapped or transformed into actual reactant molecules by adding back missing atoms or functional groups (leaving group attachment). However, whether two-stage or edit-sequence variants, miss-identifying the set of bonds that changed in the forward reaction can significantly derail downstream steps.

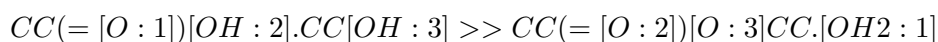
2 Data and Representation

2.1 Dataset

Our model is trained and tested on USPTO-50k (United States Patent and Trademark Office), a subset and preprocessed version of Chemical reactions from US patents (1976-Sep2016) by Daniel Lowe. It includes 50K randomly selected reactions that were later classified into 10 reaction classes by Nadine Schneider et al.^{[1][2]} It is one

of the most widely recognized datasets in ML assisted Organic Synthesis, particularly for reaction prediction and single step retrosynthesis.

USPTO-50k’s chemical reaction data come in the form of SMILES (Simplified Molecular Input Line Entry System), a chemical notation that is used to represent chemical structures and their reactions in the form of strings,



where each participating molecule is separated by a dot (.), and reactants are separated from the products by the double "greater-than" symbol (>>).

A crucial detail in the above representation that makes SMILES a suitable format for the center identification task is that reactants and products are atom mapped (:1, :2, etc.), meaning that we are able to pinpoint the exact correspondence between the atoms on both sides of a SMILES reaction.

2.1.1 Data Cleaning

Another thing that should be noted when it comes to our dataset, is that we’re using one of its rawest available versions, which contains a lot of unneeded information and noise.

To create a well-defined learning problem, we preprocess the dataset:

- We exclude reactions that produce salts (ionic compounds with no bonds)
- We exclude reaction that yield multiple products.
- We remove samples with inconsistent atom-mappings, parsing errors and invalid stereochemical properties

Focusing our task on single, well-defined products that makeup the vast majority of our target molecules

2.1.2 RDKit

Although SMILES strings themselves contain all the information that is required to extract the features of each molecule, it is a process that requires domain specific knowledge on how to parse them. RDKit is a collection of cheminformatics and machine-learning software written in C++ and Python3 whose tools offer abstraction in converting SMILES strings to Python Chemical Objects. These objects provide us with methods to directly access our model’s atom / bond features, as well as perform some predefined validation checks that ensure the integrity of our dataset (samples where multiple atoms are mapped to the same number, missing atom mappings, etc.).

In addition to that, we use RDKit’s visualization tools to produce our molecule related figures.

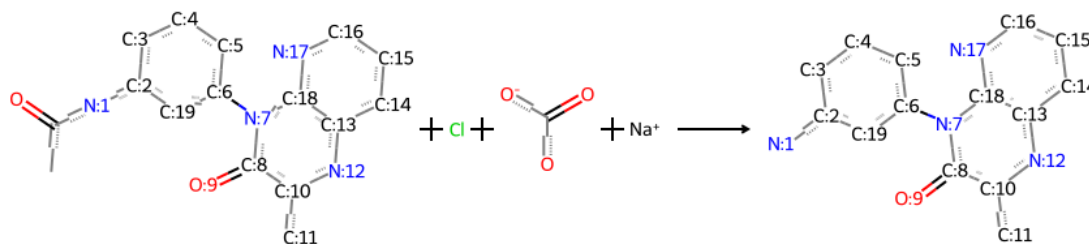


Figure 1: Visualization of a chemical reaction via RDKit

2.2 Graph Representation

Our pipeline processes the USPTO-50k dataset, converting each reaction’s product SMILES string into a graph object using RDKit and PyTorch Geometric. Graph based representations have been dominating the scene in recent years and for good reasons:

- Atoms are nodes and bonds are edges. It is a natural representation of molecular structure.
- Sidesteps strict grammatical rules enforced by SMILES strings that require rigorous navigation.
- Easier integration and learning of an atom’s local and global chemical structure

2.3 Feature Engineering & Labeling

Edge and node features are concatenated separately into vectors and fed to the model as tensors. Notably, we include the map indices only to identify corresponding atoms across reactants and products during label generation – we do not feed atom mapping IDs into the model.

Atom Features: For each atom node, we include:

- Atom type: We embed the atom’s chemical symbol as a one-hot vector over a predefined set of elements. Our task is in the context of organic chemistry. With a standard list of prevalent elements: C, N, O, S, F, H, Si, P, Cl, Br, Li, Na, K, Mg, B, Sn, I, Se, plus an "unknown" category for any others.
- Formal charge: Integer charge of the atom (0, +1, -1, etc., included as an integer).
- Degree: The number of bonds the atom has **in the product**.
- Hydrogen count: Total number of implicit or explicit hydrogens attached.
- Valence: Total valence electrons count (as computed by RDKit).
- Aromaticity flag
- Ring flag

These features capture basic chemistry (element type), local structural context (degree, H count) and chemical context (aromatic, in-ring) important for bond stability.

Bond Features:

- Bond type: One-hot encoding of bond order/type (Single, Double, Triple, or Aromatic).

- Conjugation flag
- Ring flag

These features describe the nature of the bond. For example, double bonds or aromatic bonds are less likely to be the ones breaking in many reaction types (unless it’s a specific elimination or conjugate addition scenario), whereas single bonds especially connecting functional groups are common break points.

Ground Truth Labeling: We determine the ground truth by finding the set difference between the bonds in the product and the bonds in the reactants. Any bond present in the product but absent in the reactants is labeled as a "bond to break" (positive class, label 1). All other bonds are the negative class (label 0).

2.4 Key Challenges

There are several challenges in this formulation:

- **Class imbalance:** The vast majority of bonds in a product are not reaction centers. Product molecule might have dozens of bonds, but usually only 1 or 2 of them is a reactive bond, resulting in a highly imbalanced classification problem. To be more specific, positives make up 2.86% of the dataset’s bonds. Or, in other words, there are about 34 negatives for every 1 positive.
- **Contextual decision-making:** Whether a bond is breakable often depends on the larger context of the molecule not just on local bond features. The model needs to consider aggregate information from the molecular graph when making a prediction.
- **Counting and consistency:** In our dataset almost all reactions have one or two reaction centers. If a reaction has two center bonds, the model should ideally predict both of those bonds and know that it’s a two-break reaction. Similarly, if it’s a one-bond reaction, the model should avoid false positives on additional bonds.
- **The base smiles representation lacks information about stereometry.** 2D and 3D information is overlooked.

3 Architecture

We design a multi-task GNN that simultaneously trains bond-level and molecule-level prediction tasks. We hypothesize that predicting the number of disconnections (which is essentially the number of reactants minus one, with our assumption of a single product) provides helpful global context to the bond-level task. For example, if the model can infer that two bonds likely break (e.x. ring-opening), it can adjust its bond break predictions accordingly.

3.1 Algorithm

What follows is the main flow of our implementation.

Algorithm 1 Loading the dataset

```
load the USPTO-50k csv
pairs  $\leftarrow []$ 
for each row in the csv do
    retrieve the reaction column
    split the SMILES string on >>
    split the resulting substrings on .
    pairs  $\leftarrow$  reactants, product as RDKit Mol objects
end for

for each pair in pairs do
     $x \leftarrow$  product_atom_features
    map Atom-Mappings to valid indexes
    bonds_to_break  $\leftarrow$  bonds that exist in the product, but not in the reactants
    edge_indexes, edge_attributes, edge_labels  $\leftarrow [], [], []$ 
    for each bond in the product do
         $i, j \leftarrow$  start-end atom indexes
        features  $\leftarrow$  product_bond_features
        label  $\leftarrow 1$  if bond  $\in$  bonds_to_break, 0 otherwise
        append to corresponding lists
    end for

    return graph data
end for
split the graph data to 80% train, 10% validation, 10%test
```

Algorithm 1 describes the loading and processing of the dataset.

Once the dataset has been parsed, processed and loaded, the training process starts:

1. Calculate *negative_sample/positive_sample* ratio to feed it to the Loss function
2. Initialize the **optimizer**
3. Initialize the learning rate **scheduler**
4. Begin the training loop for each batch
 - (a) Feed the batch to the **model** and get:
 - A prediction for every single bond (breaks / doesn't break)
 - A prediction on how many bonds should be broken
 - (b) Calculate the aggregated **Loss Function** of the two tasks
 - (y) Update the model's parameters
5. Evaluate current model state on the **validation dataset metric**
6. Update learning rate based on the **validation dataset metric**

After training, the model is evaluated on the test set using a Beam Search strategy.

- Instead of rigidly adhering to the count-head's prediction, we generate a list of candidate disconnection sets.

- Each hypothesis is scored and ranked based on the combined probabilities from, both the bond-head and count-head.
- We measure Top-k accuracy by checking if the ground truth disconnection set is present in the top k hypotheses generated by the beam search.
- A Precision-Recall curve for the bond binary classification task
 - Is the bond part of the product’s Reaction Center?
- A Confusion Matrix for the Reaction Center’s cardinality Classification Task
 - How many bonds are a part of the product’s Reaction Center?

3.1.1 The Model

Our model’s architecture is showcased in Figure 2 and it can quickly become evident that it is comprised of 4 main modules:

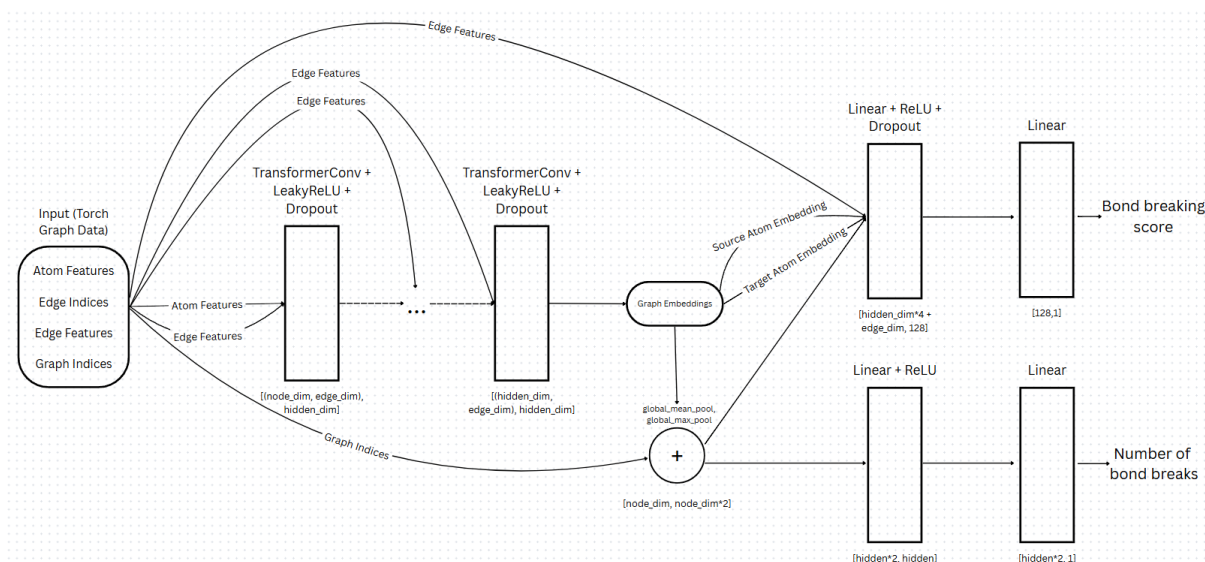


Figure 2: Our model

- **Graph Encoder:** It is the Graph Neural Network of the pipeline. It implements multiple layers of PyTorch Geometric’s TransformerConv [7] model to convert Graph Data into context-rich atom embeddings (through the Message Passing mechanism), representing information contained in:
 - The atom’s individual features
 - The neighboring atom’s features
 - The atom’s connected edges’ features

TransformerConv is an Attention-Based Graph Convolution operator that creates an abstraction over Message-Passing. In more detail, for each individual atom it:

1. Calculates the multi-head attention coefficients for each of its neighbors, enhanced with the features of the bond between them:

$$\alpha_{i,j} = \text{softmax} \left(\frac{(\mathbf{W}_3 \mathbf{x}_i)^\top (\mathbf{W}_4 \mathbf{x}_j + \mathbf{W}_6 \mathbf{e}_{ij})}{\sqrt{d}} \right)$$

- Creates the atom embedding as a weighted sum of the atom’s features and its neighbor’s features (+ the features of the bond between them) multiplied by said coefficients:

$$\mathbf{x}'_i = \mathbf{W}_1 \mathbf{x}_i + \sum_{j \in \mathcal{N}(i)} \alpha_{i,j} (\mathbf{W}_2 \mathbf{x}_j + \mathbf{W}_6 \mathbf{e}_{ij})$$

Concerning the Message-Passing implementation that we’ve seen in GNNs, no extra work was needed from our side since the model handles it automatically by parsing a given edge index list.

- **Global Feature Extraction:** One of the most crucial factors in the improvement of the model’s performance is the calculation and injection of graph-level features into the prediction tasks. In our model, this is achieved by passing each individual graph’s atom embeddings as input to PyTorch’s *global_mean_pool* and *global_max_pool* functions. As simple as it may seem, aggregating a graph’s embeddings to produce new ones (of the same dimensionality) has proven to be quite effective.

$$\mathbf{r}_i = \frac{1}{N_i} \sum_{n=1}^{N_i} \mathbf{x}_n, \quad \mathbf{r}_i = \max_{n=1}^{N_i} \mathbf{x}_n.$$

- **Bond-Head Prediction:** A 2-layer MLP that takes the graph’s atom embeddings at a given edge index, and the graph’s edge features and the graph-level features as input, and produces a ”score” that indicates the likelihood of the bond at the given edge index belonging to **any** Reaction Center.
- **Count-Head Prediction:** A 2-layer MLP that takes the graph’s global features as its input and produces a ”score” corresponding to the likelihoods that a molecule has 0,1,2 bonds as its Reaction Center. During the upcoming inference stage, this acts as a ”soft prior” to guide the beam search, rather than a hard rule.

3.1.2 Beam Search Algorithm

The Beam Search algorithm acts as a post-processing step that leverages the outputs of both the Bond-Head and the Count-Head to generate and rank the most probable sets of reaction center bonds. This overcomes the limitations of rigid argmax predictions from the Count-Head, especially when the true number of breaks is underestimated. Let L_B be the bond logits produced by the Bond-Head for all unique bonds in the product molecule, and $L_C = [l_{C,0}, l_{C,1}, l_{C,2}]$ be the count logits from the Count-Head for 0, 1, or 2 reaction centers, respectively.

The algorithm proceeds as follows:

Calculate Bond Log-Probabilities: For each unique bond b_i in the product molecule, we convert its raw logit $L_{B,i}$ into a log-probability using the sigmoid function:

$$\log P(b_i \text{ breaks}) = \log(\text{sigmoid}(L_{B,i}))$$

Calculate Count Priors: The count logits are converted into probabilities using the softmax function, providing a prior distribution over the number of reaction centers:

$$P(\text{count} = k) = \frac{\exp(l_{C,k})}{\sum_{j=0}^2 \exp(l_{C,j})}$$

From these, we derive the log-priors:

$$\log P(\text{count} = k) = \log(P(\text{count} = k) + \epsilon)$$

where ϵ is a small constant (e.g., 10^{-9}) to prevent $\log(0)$ issues.

Generate and Score Hypotheses: We generate a set of candidate hypotheses, H , where each hypothesis $h \in H$ is a set of unique bonds. Each hypothesis is assigned a score $S(h)$ that combines the count prior with the individual bond probabilities. A weighting factor w is used to control the relative influence of the individual bond probabilities versus the count prior.

Hypothesis for 0 breaks (h_0): This hypothesis represents no bonds breaking.

$$H_0 = \{\emptyset\}$$

$$S(h_0) = \log P(\text{count} = 0)$$

Hypotheses for 1 break (h_1): For each unique bond b_i , we form a hypothesis $\{b_i\}$. The score for such a hypothesis is:

$$S(\{b_i\}) = \log P(\text{count} = 1) + w \cdot \log P(b_i \text{ breaks})$$

We consider the M_1 bonds with the highest $\log P(b_i \text{ breaks})$ values to generate these hypotheses. M_1 is typically a multiple of the beam size (e.g., $2 \times \text{beam_size}$).

Hypotheses for 2 breaks (h_2): For every unique pair of distinct bonds (b_i, b_j) , we form a hypothesis $\{b_i, b_j\}$. The score for such a hypothesis is:

$$S(\{b_i, b_j\}) = \log P(\text{count} = 2) + w \cdot (\log P(b_i \text{ breaks}) + \log P(b_j \text{ breaks}))$$

We consider pairs from the top M_2 bonds with the highest individual $\log P(b_i \text{ breaks})$ values. M_2 is usually the same as M_1 .

Rank and Select Top Hypotheses: All generated hypotheses are collected and sorted in descending order based on their scores $S(h)$. The top k hypotheses, where k is the `beam_size`, are then selected as the final ranked predictions.

$$H_{\text{ranked}} = \text{TopK_by_Score}(H, k)$$

This method ensures that even if the Count-Head’s most confident prediction for the number of breaks is incorrect, a hypothesis with the true number of breaks can still achieve a high combined score and appear among the top predictions, especially if the individual bonds in that true set have high probabilities from the Bond-Head. This greatly improves the model’s ability to find the correct reaction center in a Top-k evaluation framework.

3.1.3 Frameworks, Techniques, Metrics used

- **Optimizer:** For our optimizer we use AdamW, a standard choice in Deep Learning pipelines.
- **LR scheduler:** We also use a Learning Rate scheduler in another attempt to tackle the task’s complexity. In our pipeline we use CosineAnnealingLR, which adjusts the Learning Rate according to the cosine curve, i.e. it starts with a high LR to take big steps towards the optimum, and slowly decreases it to a minimum value for more refined updates, which are presumed to be closer to optima.

- **Loss Function:** Due to the double-task nature of our model, our Loss Function is an aggregation of two separate losses:
 - **Binary Cross Entropy** for Reaction Center Prediction
 - **Cross Entropy** For Reaction Center Set cardinality prediction

$$Loss = L_{bondBCE} + * L_{head_countCE}$$

- **Bayesian-Optimization / Hyperparameter Tuning**
Due to the complexity of our task and in order to get the best out of our model, we quickly turned to the integration of an automatic Hyperparameter Optimizer, **Optuna**^[7]. We ran Optuna on several hyperparameters and technique alternatives for 50 trials, each one training the model for 100 epochs. The results can be found further down the report in the "Results" section

4 Benchmarking

4.1 Baselines

4.1.1 A weaker model

Reaction Center prediction is adopted as a module in numerous Deep Learning Retrosynthesis related works (single-step and multi-step Retrosynthesis alike). However, hardly any of them provide evaluation metrics for it as an individual task in the retrosynthesis pipeline, essentially leaving us with no proven benchmarks or baselines. In the wake of this, we chose our baseline for this project to be one of our earliest attempts at a Reaction Center prediction pipeline (one that actually learn and output respectable results), and made it our goal to explore techniques and frameworks to find out how much we can improve on it.

The baseline’s differentiations from our final work can be summed up as:

- **Multi-task prediction Beam Search:** While the baseline model implements much of the same logic as our final one, it is missing the bond counting task completely. Of course, this means that the model is unable to make accurate predictions for graphs whose Reaction Center contains 2 bonds.
- **Graph-level / global features:** We relied solely on atom embeddings to make predictions.
- **Learning Rate scheduler**
- **Hyperparameter optimization:** We initially ran the baseline model with standard hyperparameter values, or with little empirical tuning.

4.1.2 G2G Center Identification

One of the few publications that evaluate the Center Identification module as a standalone module comes from Chence Shi et al.[8], in which the authors propose a GIN encoder (Graph Isomorphism Network), as opposed to our Transformer-Based approach . You may find their results along with ours further down in the "Results" section.

4.2 Metrics

AUPRC and PR-curve: Validation & Test

Considering our dataset’s massive imbalance against the positive class (i.e. bonds that are part of a Reaction Center), we use **Area Under the PR Curve** (AUPRC) as our validation and primary metric for the bond-prediction task. We want to force our model to focus on finding the rare positive instances of the dataset (2.86% of the dataset’s bonds).

Top-k Accuracy: Test

We evaluate our model’s performance on the test set in a Top- k framework, meaning the model can propose up to k different disconnection sets (like a ranked list of guesses) and we count it as correct if the true answer is in that list. This simulates a retrosynthetic planner generating multiple suggestions and the correct one being present. Top-1, 3, 5, 10 Accuracy is also the standard evaluation metrics for Retrosynthesis models.

Count Prediction: Test

The number of times the model correctly predicted the number of bonds to break. A good indicator of the Counting Head’s performance.

ROC curve: Test

While we don’t expect ROC AUC to offer as useful information as the other metrics because of the negative class dominating the dataset, we showcase its results in comparison to the PR curve.

4.3 Training-Validation-Test pipeline

The cleaned up version of the USPTO-50k dataset contains 48232 total samples, out of which 38585 were used for training ($\sim 80\%$), 4823 for validation ($\sim 10\%$) and 4824 for testing ($\sim 10\%$). On these data splits, the models were:

- Trained for 25 epochs, with the potential of early stoppage after 8 consecutive epochs with no improvement on the validation set’s AUPRC.
- Evaluated on the validation set. Out of all the epochs, only the model instance that resulted in the highest AUPRC was kept.
- Evaluated on the test set.

5 Results

5.1 Benchmarks

Table 1: Comparison of Top- k accuracy (%) across models

Model / Setting	Top-1	Top-2	Top-3	Top-5	Top-10
Baseline	46.0	–	67.3	69.9	70.5
G2G ^[8] (reaction class known)	90.2	94.5	94.9	95.0	–
G2G ^[8] (reaction class unknown)	75.8	83.9	85.3	85.6	–
Our best model(reaction class unknown)	64.9	–	91.5	95.8	98.4

Our model **outperforms G2G** (reaction class unknown) once $k \geq 3$. While G2G still has a higher Top-1 (75.8 % vs. 64.9 %), our model overtakes at Top-3 and beyond, reaching 91.5 % / 95.8 % / 98.4 % compared with G2G’s 85.3 % / 85.6 % / –.

We also highlight that our model displays a steeper accuracy curve. The jump from Top-1 to Top-3 is +26.6 pp (64.9 $\rightarrow$ 91.5), almost matching G2G-unknown’s total improvement all the way to Top-5 (+9.8pp). This sharp gain implies that our ranking mechanism places variants of the correct center immediately after the first guess, improving downstream reranking or ensemble scenarios.

5.2 Baseline

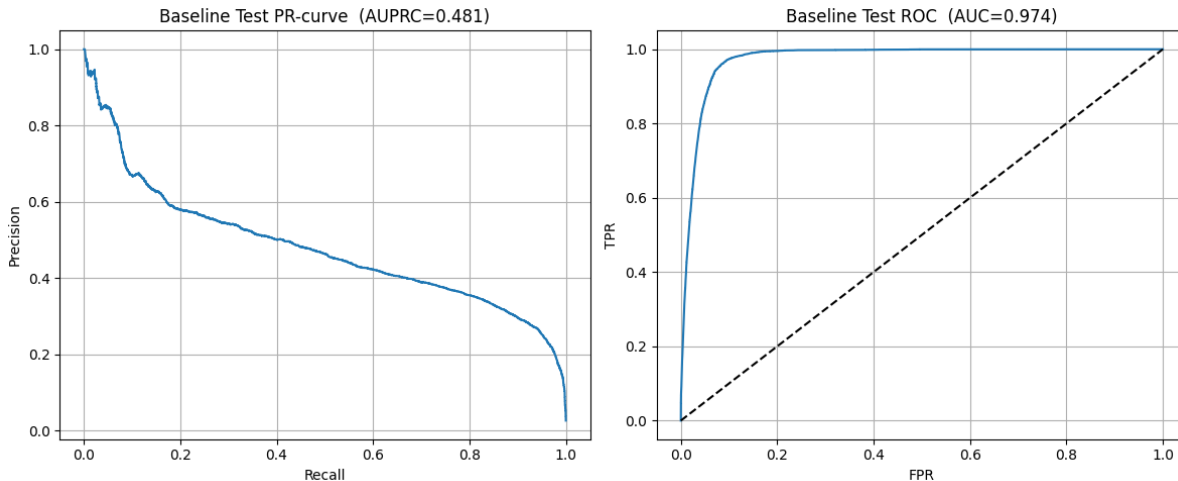


Figure 3: Baseline Precision-Recall Curve and AUPRC

Precision-Recall Curve and AUPRC: The baseline achieves an AUPRC of ≈ 0.481 on the test set. As an indicator, the depicted curve in Figure 3 indicates that $\sim 60\%$ precision comes with the tradeoff of $\sim 20\%$ recall, while $\sim 60\%$ recall means $\sim 40\%$ precision.

ROC AUC Curve: A ROC AUC of ~ 0.974 here is a great indicator as to why this metric is not useful for our task. Whilst our baseline model is clearly under-performing, its ROC AUC is high because the model can successfully predict almost all of the $\sim 97\%$ negative samples.

Top-k Accuracy

- Top-1 : 46.0%
- Top-3 : 67.3%
- Top-5 : 69.9%
- Top-10 : 70.5%

Train & Validation Loss

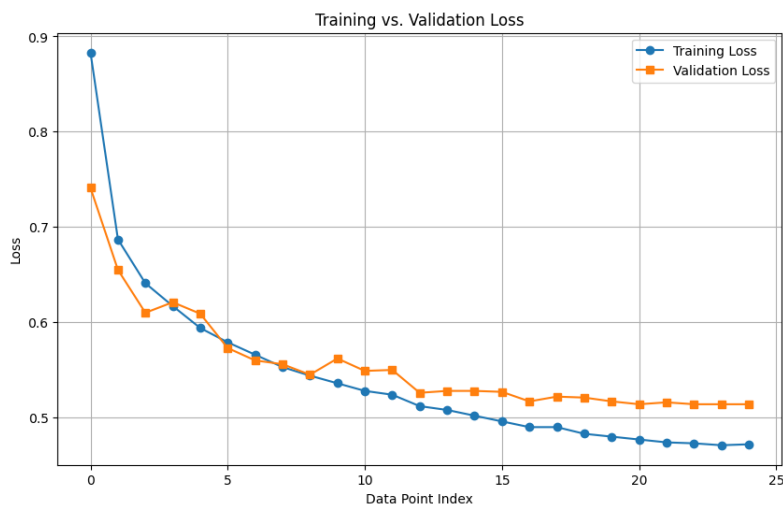


Figure 4: Baseline Train and Validation Loss

5.3 Our Best Model

5.3.1 Final parameters

- Hidden Dim: 128 - Empirical
- Layers: 2 - Empirical
- Dropout: 0.2 - Empirical
- Starting Learning Rate: 0.00099040204749014 - Optuna
- Lambda (Count Head Loss Factor): 0.5665524341505498 - Optuna
- Scheduler: CosineAnnealingLR - Optuna
- Epochs: 25
- Early stop after: 8

- Batch size: 32 - Empirical
- Beam Search: True

Bond Break Prediction Accuracy: It's not very informative to report "accuracy" per bond because of imbalance. The model could be $> 97\%$ accurate by labeling everything as non-reaction, which is not useful. Instead, the PR and AUC metrics above are preferred.

ROC AUC (Receiver Operating Characteristic Area Under Curve): Our model achieved an AUC of ≈ 0.984 on the test set, indicating near-perfect ranking of true reaction center bonds above non-reactive bonds. As explained above, high AUC is partly due to the fact that many negatives are trivial to identify. Still, it shows the model has learned a strong signal.

Precision-Recall Curve and AUPRC: Given the class imbalance, Precision-Recall (PR) curve is the most important metric for this task. The area under the PR curve (AUPRC) for the model is about 0.636. Based on the curve, we can achieve, for example, around 70% precision at 50% recall, or vice versa. If the model predicts 1 bond per molecule as reactive, it's correct about 70% of the time. And, if we allow it to predict a few more bonds precision goes down but recall approaches covering most true cases.

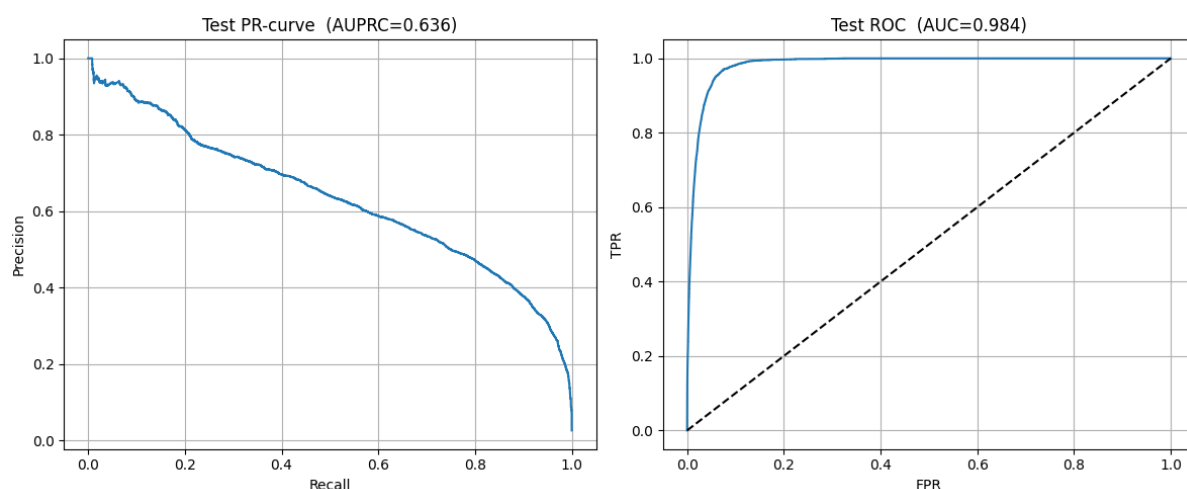


Figure 5: Precision-Recall Curve and AUPRC.

Count Prediction The overall accuracy of predicting the correct number of bonds to break was 87.2% on test. Most reactions in the dataset involve only a single bond breaking and it contains very few two-center reactions (89 in total), so most mass lies on the diagonal. Overall, there is a slight downward estimation bias. Overestimation adds an extra bond to the candidate set which lowers precision but does not prevent the correct route from being in top-k. Conversely, underestimation which happens about $57/89 \approx 64\%$ of the time for double bond break is catastrophic. This is why we rely on beam search using this as a prior and not a hard argmax.

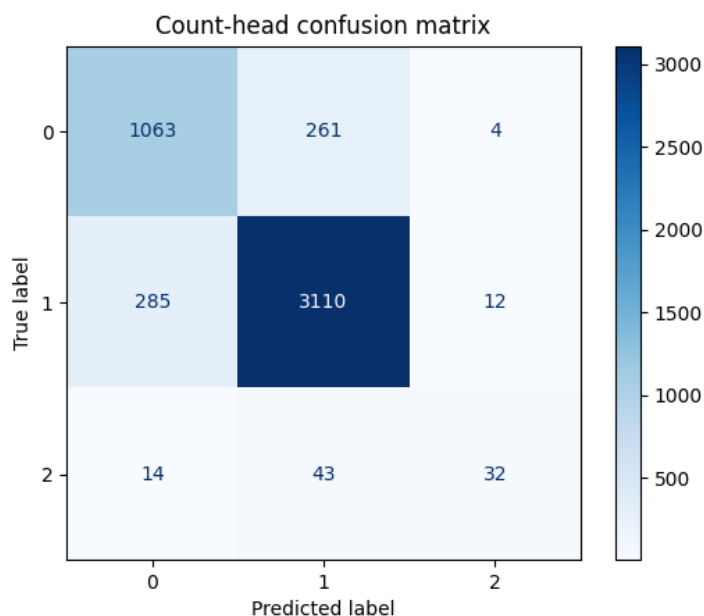


Figure 6: Confusion matrix of the multi-task *count head* (0/1/2 predicted bond breaks).

Top- k Accuracy

- Top-1 : 64.9%
- Top-3 : 91.5%
- Top-5 : 95.8%
- Top-10 : 98.4%

Train & Validation Loss

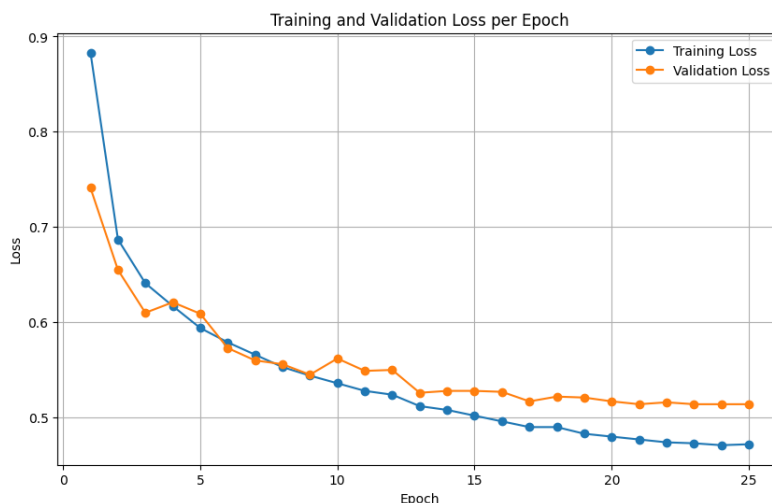


Figure 7: Training and Validation Loss

Qualitative Examples on Test Set:

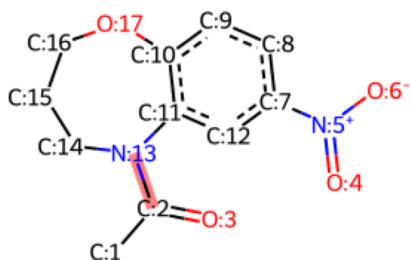
Success:

- For each, the highest scoring bond (Top-1) coincides with the ground-truth center.
- Typical motifs include C-N and C-O bonds adjacent to carbonyls or hetero-aromatic rings, matching textbook disconnections.

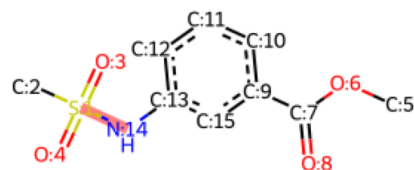
Failures:

- *Over-confident false positive* on highly activated C-O . Context about the leaving group could improve performance on such cases.
- *Alternative valid disconnection*: the model picks a plausible ester cleavage that is not the one recorded in the patent

True: [(np.int64(1), np.int64(3))] | Pred: [(np.int64(1), np.int64(3))]



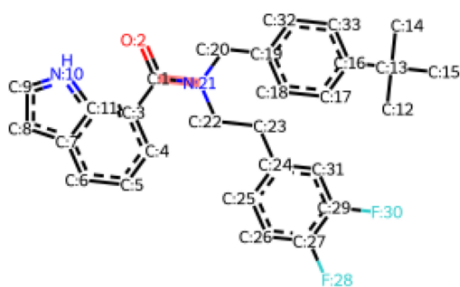
True: [(np.int64(0), np.int64(4))] | Pred: [(np.int64(0), np.int64(4))]



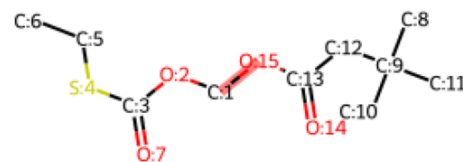
(a) Correctly identified C-N amide bond break.

(b) Correctly identified S-N sulfonamide bond break.

True: [(np.int64(0), np.int64(11))] | Pred: [(np.int64(0), np.int64(11))]



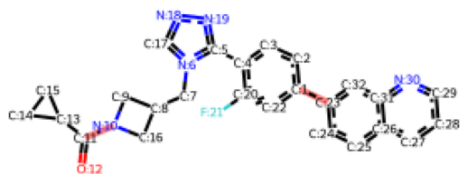
True: [(np.int64(0), np.int64(7))] | Pred: [(np.int64(0), np.int64(7))]



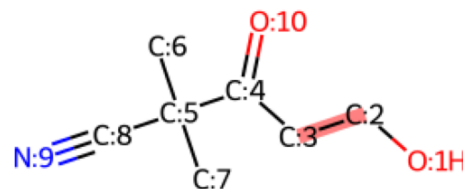
(c) Correctly identified C-N amide bond break.

(d) Correctly identified ester C-O bond break.

True: [(np.int64(0), np.int64(1))] | Pred: [(np.int64(19), np.int64(20))]



True: [] | Pred: [(np.int64(1), np.int64(2))]



(e) Incorrect bond, correct number of centers.

(f) Incorrect number of centers.

Figure 8: Example predictions from the test set. The ground truth bonds are listed as "True" and the model's prediction as "Pred," with the corresponding bond highlighted in the molecule visualization. The examples show a mix of successful (a, b, c, d) and unsuccessful (e, f) predictions.

6 Experimentation

6.1 Ablations

6.1.1 Beam Search

Beam Search is not a direct improvement to the model itself, but implementing and utilizing in conjunction with the Counting Head results turned out to be the biggest factor in our pipeline’s final results when referring to its Top-k evaluation.

Without the use of Beam Search, the pipeline’s 2 predictions were extremely dependent on the Counting Head module:

1. The model produced a Counting Head prediction
2. Based on the Counting Head prediction, it searched specifically for bond sets of 0 / 1 / 2 bonds.

This clearly limits the entire model since a wrong counting head prediction guarantees a 100% wrong Reaction Center prediction. However, implementing beam search to retrieve the most probable Reaction Center bonds and using the Counting Head’s outcome as a prior probability to them, turned out to be a massive performance booster.

	Top-1	Top-3	Top5	Top10
No Beam Search	47.0%	68.0%	70.2%	71.0%
With Beam Search	64.9%	91.5%	95.8%	98.4%

Table 2: Beam Search ablation test results

The difference is quite remarkable, with Top-1 accuracy improving by about 18%, Top-3 and Top-5 23.5% and 25.6% respectively, and finally Top-10 with an $\sim 28.4\%$ improvement, close to 100% Accuracy.

6.1.2 No global features (& Multi-task)

Another significant part of the pipeline concerns the global features extracted. They are by far the most impactful features used, something that makes sense intuitively as well, considering they capture the whole graph’s characteristics. For our implementation however, their presence becomes of utmost significance when we consider that they are the only features fed into the Counting Head task, rendering it unusable if they’re removed.

Setting	AUPRC	Top-1	Top-3	Top5	Top10
No Global (& No Multi-Task)	0.477	46.0%	64.9%	68.8%	71.0%
No Multi-Task	0.476	46.4%	65.0%	69.9%	70.5%
Global & Multi-Task	0.636	64.9%	91.5%	95.8%	98.4%

A $\sim 16\%$ dropoff in AUPRC, $\sim 19\%$ in Top-1, $\sim 26 - 27\%$ in Top-3, Top-5 and Top-10 confirm the importance of both the Graph-level features, as well as the existence of the Counting Head module in our model.

6.1.3 Number of Hydrogens

In the process of continuously improving our model through different trials and implementations, an ablation study was carried out (on one of our most stable versions) for each individual atom / bond feature that we’re using. Most of the features seemed to be of little significance to the end results, possibly due to overlap with the rest of the features or even the complexity of the model.

The two only features that stood out as ones that had a noticeable impact on the models AUPRC were the atoms’ Symbol and more so the Total Number of Hydrogens. Having kept that in mind, we also decided to explore the atom’s Number of Hydrogens impact in our final version of the model as well. The results confirm our previous observations.

Setting	AUPRC	Top-1	Top-3	Top5	Top10
	AUPRC	Top-1	Top-3	Top5	Top10
No Hydrogens	0.600	61.7%	89.1%	94.4%	98.3%
With Hydrogens	0.636	64.9%	91.5%	95.8%	98.4%

The impact is minimal if we look at Top-3, Top-5 or Top-10 accuracies. However a quite noticeable $\sim 3\%$ dropoff is observed in Top-1 Accuracy, as well as a $\sim 3.5\%$ one on the model’s AUPRC in the test set.

7 Conclusion

In our work we have achieved the following

- **Effective Multi-Task GNN:** We successfully developed and implemented a multi task GNN architecture that simultaneously predicts individual bond break probabilities and the total number of breaking bonds, leveraging atom, bond, and global graph features.
- **Critical Beam Search:** We developed and implemented a beam search algorithm during inference that enabled our model to take full advantage of our multi task formulation. Allowing the model to explore and rank multiple plausible sets of reaction centers by utilizing graph level features to the fullest without being inhibited by limitations of rigid count predictions.
- **Strong Performance:** Our model achieved a **0.636 AUPRC** and high Top-k accuracies on USPTO-50k: **64.9% Top-1, 91.5% Top-3, 95.8% Top-5**, and **98.4% Top-10**. Even outperforming a G2G baseline for $k \geq 3$ without using reaction class information.
- **Validated Ablations:** Utilized ablation studies to confirm the essential contributions of global features, multi-task setup, beam search, and atom-level hydrogen count to the model’s success.

7.1 Future Improvements

Integrating Reaction Generation: The next step after identifying a bond to break is to actually predict how the resulting fragments become reactants.

- A simple extension is to train a second model that takes the product + identified reaction center and predicts the full reactant structures. However, recent literature has shown that training models separately for these tasks is not optimal. [5]
- Perform reaction center identification and synthon completion in one **end-to-end model**. This could take the form of a multi task predicting leaving groups along with reaction center with a semi-template informed approach [5]. Or a graph generation approach training the two tasks together without any template information, for example using auto regressive graph editing.

Additional Features: We could incorporate more chemistry knowledge and or stereometry (graph level geometry) information in the features.

- Using calculated properties such as bond enthalpy estimates, or atomic partial charges, or identifying functional group types as categorical features. Our model did well with basic features, but these additions might help in the borderline cases [Retro-MTGR]
- Due to the limitation of SMILES representations, there is a distinct lack of higher level information regarding the geometry of compounds in the data. However, this geometry plays a major role in reactions. Including information on 2D and 3D stereometry, by utilizing models that synthesize this information based solely on RDKit molecules as input has been shown to have great effect in recent papers like [10].

Data Augmentation:

- Synthetic duplication of multi-centre reactions (or adding external ring-forming samples) can give the head more signal for the rare class.
- Introducing symmetry and ordering information into our data along with appropriate pre-training/warmup on a task for distinguishing could yield significant improvements [11].

Transfer learning and larger data: The USPTO-50k dataset is relatively small. Scaling to larger datasets like the full MIT-USPTO with more than a million reactions could improve performance as the model sees more varied chemistry. However, larger data also brings more noise. Pre-training on a larger set and fine-tuning on a high-quality set could be a strategy.

Reaction Class Awareness: We ignored the reaction class labels in USPTO-50k. In practice, knowing the class can significantly narrow down which bonds are likely reactive. A possible extension is a model that first classifies the reaction type (or does so jointly) and uses that information to inform the reaction center prediction (e.g., via a conditional network or by simply providing the class as an input feature). This might boost accuracy, since certain classes have very characteristic bond changes (e.g., in an oxidation, we know a certain bond type changes).

8 References

[1] USPTO-50k, <https://paperswithcode.com/dataset/uspto-50k>

[2] Nadine Schneider, Nikolaus Stiefl, Gregory A. Landrum (2016). - What's What: The (Nearly) Definitive Guide to Reaction Role Assignment, Journal of Chemical Information and Modeling <https://pubs.acs.org/doi/pdf/10.1021/acs.jcim.6b00564>

[3] RDKit <https://www.rdkit.org/>

[4] Investigating the reliability and interpretability of machine learning frameworks for chemical retrosynthesis, <https://pubs.rsc.org/en/content/articlehtml/2024/dd/d4dd00007b#cit>

[5] (2025) Single-step retrosynthesis prediction via multitask graph representation learning, <https://www.nature.com/articles/s41467-025-56062-y>

[6] TransformerConv, PyTorch Geometric Documentation <https://pytorch-geometric.readthedocs.io>

[7] Optuna, An open source hyperparameter optimization framework to automate hyperparameter search <https://optuna.org>

[8] Chence Shi, Minkai Xu, Hongyu Guo, Ming Zhang, Jian Tang, A Graph to Graphs Framework for Retrosynthesis Prediction <https://arxiv.org/pdf/2003.12725>

[9] Node-Aligned Graph-to-Graph (NAG2G): Elevating Template-Free Deep Learning Approaches in Single-Step Retrosynthesis <https://arxiv.org/abs/2309.15798>

[10] Retro-MTGR: Molecule Retrosynthesis Prediction via Multi-Task Graph Representation Learning https://www.researchgate.net/publication/373725811_Retro-MTGR_Molecule_Retrosynthesis_Task_Graph_Representation_Learning

[11] UAlign <https://arxiv.org/abs/2412.11803>